

Maths for ML — Part 1

This guide builds the linear-algebra and calculus foundations behind machine learning. Run the Python examples with `numpy` installed (`pip install numpy`). Unless noted otherwise, vectors are column vectors.

Vectors and Matrices

1. Overview

A vector is an ordered list of numbers; a matrix is a rectangular grid of numbers. They represent feature records, batches, images, neural-network weights, and transformations.

2. Intuition

A vector is an arrow with direction and length, such as a customer's `[age, income, purchases]`. A matrix is a spreadsheet of many such records or a machine that turns one vector into another.

3. Prerequisites

Arithmetic, coordinate planes, Python lists, and NumPy array shapes.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Scalar	One number, e.g. <code>3.0</code>	A scalar scales a vector.
Vector	Shape <code>(d,)</code> , e.g. <code>[2, -1]</code>	Feature vector; distinguish row vs column orientation.
Matrix	Shape <code>(m, n)</code> , e.g. 100 samples x 4 features	Rows usually samples, columns features.
Tensor	General n-dimensional array	Image batches are <code>(N, C, H, W)</code> .
Shape	Number of entries per axis	Most ML bugs are shape/broadcasting bugs.

5. Algorithm / Working Process

1. Encode one observation as a feature vector.
2. Stack observations into a data matrix `X`.
3. Standardize or transform columns when needed.
4. Feed `X` to a model that produces predictions per row.

6. Mathematical Foundation

For $x, y \in \mathbb{R}^d$ and scalar c : $x + y = [x_i + y_i]$, $cx = [cx_i]$. A matrix $A \in \mathbb{R}^{(m \times n)}$ has m rows and n columns. In linear regression, $X \in \mathbb{R}^{(N \times d)}$, weights $w \in \mathbb{R}^d$, and predictions have shape N .

7. Practical Implementation

```
import numpy as np

x = np.array([2.0, -1.0, 3.0])      # one sample, shape (3,)
X = np.array([[2., -1., 3.], [0., 4., 1.]]) # two samples, shape (2, 3)
scaled = 0.5 * x
column_means = X.mean(axis=0)
print(x.shape, X.shape, scaled, column_means)
```

8. Code Explanation

`x` holds three features. `X` stacks two examples by rows. `axis=0` aggregates down rows, yielding one mean for each feature column.

9. Training / Evaluation

Prepare `X` with one row per example and consistent feature order. Fit preprocessing only on training rows, then reuse it for validation/test rows to prevent leakage. Shapes, missing values, and feature scales matter more here than model metrics.

10. Complexity and Cost

Storing an $N \times d$ float32 matrix costs $4Nd$ bytes. Elementwise operations cost $O(Nd)$ and run efficiently on CPUs/GPUs.

11. Common Use Cases

Tabular feature tables, token embeddings, pixel arrays, recommendation embeddings, and batches of model inputs.

12. Common Mistakes

- Mixing samples-as-columns with samples-as-rows.
- Broadcasting an incompatible shape silently.
- Fitting a scaler on all data before splitting.
- Using integer arrays when fractional calculations are required.

13. Edge Cases / Limitations

Dense matrices waste memory for mostly-zero text features or graphs; use sparse representations. Raw feature vectors also do not automatically capture sequences or spatial structure.

14. Variations

Dense vs sparse matrices (sparse for high-dimensional zeros); embeddings (learned dense vectors); tensors (images/video). All are placement-important.

15. Related Topics

Dot products score vectors; matrix multiplication applies layers; norms measure size; tensors generalize matrices in PyTorch.

16. Interview Questions

1. **What is a vector?** An ordered numeric representation of a point, feature set, or direction.
2. **What does (100, 20) usually mean for $\mathbf{x}$?** 100 samples and 20 features.
3. **Vector vs scalar?** A vector has multiple components; a scalar has one.
4. **Matrix vs tensor?** A matrix is 2-D; a tensor may have any number of axes.
5. **Why do shapes matter?** They define valid operations and model interfaces.
6. **What is a feature matrix?** Rows are examples and columns are measured/derived features.
7. **Why use float32 in deep learning?** It halves memory versus float64 and is usually accurate enough.
8. **What is broadcasting?** NumPy/PyTorch expand compatible size-1 axes during elementwise operations.
9. **Why standardize columns?** To make magnitudes comparable for scale-sensitive models.
10. **How store one-hot labels?** As a vector of length classes, or more often integer class IDs.

17. Practice Tasks

- Create an (8, 3) feature matrix and compute column means.
- Split a matrix into train/validation rows without leakage.
- Diagnose a failing operation caused by shapes (10, 3) and (10,).
- Convert a grayscale image array into a normalized float tensor.

18. Project Ideas

Project	Stack / data	Resume value
Feature-quality dashboard	Pandas, NumPy, Titanic/UCI data	Demonstrates data-shape and preprocessing discipline.
Image tensor explorer	NumPy, OpenCV, CIFAR-10	Shows image representation knowledge.
Embedding search demo	NumPy, sentence embeddings, FAQ data	Connects vectors to semantic retrieval.

19. Quick Revision

Key idea: vectors hold features; matrices stack records or weights. **Formula:** $\mathbf{X} \in \mathbb{R}^{(N \times d)}$. **Use:** virtually every ML pipeline. **Trap:** wrong axis/shape. **One-liner:** ML is largely structured numerical arrays plus transformations.

20. Final Cheat Sheet

Definition	Input → output	Cost	Best use
Vector/matrix	numbers arranged in 1-D/2-D	storage $\mathcal{O}(Nd)$	features, batches, parameters

Matrix Multiplication

1. Overview

Matrix multiplication composes linear mappings. It powers dense neural-network layers, attention projections, covariance computations, and batched model scoring.

2. Intuition

Each output number is a weighted combination: a row of the left matrix "asks" every column of the right matrix how well it matches.

3. Prerequisites

Vectors/matrices, dot product, shapes, and summation notation.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Compatibility	$(m,n) @ (n,p)$ is valid	Inner dimensions must match.
Output shape	Result is (m,p)	Infer layer shapes quickly.
Row-column rule	$C_{ij} = \sum_k A_{ik} B_{kj}$	Explain without elementwise multiplication.
Non-commutativity	Usually $AB \neq BA$	Transformation order matters.
Associativity	$(AB)C = A(BC)$	Choose efficient computation order.

5. Algorithm / Working Process

For every output row i and column j , multiply corresponding entries from row i of A and column j of B , sum them, and store the result at $C[i,j]$.

6. Mathematical Foundation

If $A \in \mathbb{R}^{(m \times n)}$ and $B \in \mathbb{R}^{(n \times p)}$, then $C=AB \in \mathbb{R}^{(m \times p)}$ with $C_{ij} = \sum_{k=1..n} A_{ik} B_{kj}$. A dense layer is $Y=XW+b$, where $X: (N,d)$, $W: (d,h)$, $Y: (N,h)$.

7. Practical Implementation

```
import numpy as np

X = np.array([[1., 2.], [3., 4.]])      # (2, 2): batch
W = np.array([[0.5, -1.], [2., 0.25]]) # (2, 2): weights
b = np.array([0.1, 0.2])               # broadcasts across rows
Y = X @ W + b
print(Y)
```

8. Code Explanation

`@` performs matrix multiplication, unlike `*`, which multiplies element by element. Adding `b` broadcasts one bias vector to every sample.

9. Training / Evaluation

During training, learn `W` and `b` by backpropagation. Validate output shapes and normalize inputs where training is unstable. Evaluation depends on the final task, not multiplication itself.

10. Complexity and Cost

Naive $(m,n) @ (n,p)$ costs $O(mnp)$ time and $O(mp)$ output memory. GPUs accelerate large, dense products; batching reduces overhead.

11. Common Use Cases

Fully connected layers, transformer Q/K/V projections, PCA transforms, collaborative filtering, and graph neural-network message aggregation.

12. Common Mistakes

- Using `*` instead of `@`.
- Forgetting to transpose weights.
- Assuming multiplication is commutative.
- Materializing huge intermediate matrices unnecessarily.

13. Edge Cases / Limitations

Large dense products are memory-bandwidth and compute intensive. Numerical precision can degrade for badly scaled or very long sums.

14. Variations

Matrix-vector multiplication for one example; batched `matmul`; sparse `matmul` for text/graphs; block multiplication. Dense layers and attention make this placement-critical.

15. Related Topics

The dot product is one output cell. Linear transformations use multiplication; gradients of layers require transposes and chain rule.

16. Interview Questions

1. **When is AB defined?** When columns of A equal rows of B .
2. **Shape of $(32, 128) @ (128, 64)$?** $(32, 64)$.
3. **Is it elementwise multiplication?** No; it sums row-column products.
4. **Is $AB=BA$?** Generally no.
5. **Dense-layer equation?** $Y=XW+b$.
6. **Why transpose in backprop?** It routes gradients through the linear map with compatible shapes.

7. **Why GPUs help?** They parallelize many multiply-add operations.
8. **What is a batched matmul?** Multiple independent products computed together.
9. **Why does order matter?** Each matrix represents a transformation; composing in a different order changes it.
10. **How reduce cost?** Exploit sparsity, low rank, batching, or better parenthesization.

17. Practice Tasks

- Manually compute a 2×3 times 3×2 product.
- Implement nested-loop multiplication and compare with NumPy.
- Derive shapes for a 768-to-3072 transformer MLP layer.
- Benchmark dense versus sparse multiplication on a bag-of-words matrix.

18. Project Ideas

Project	Stack / data	Resume value
Neural-layer from scratch	NumPy, synthetic classification	Explains forward/backward passes.
Mini attention block	PyTorch, text embeddings	Shows transformer fundamentals.
Sparse recommender scorer	SciPy sparse, MovieLens	Demonstrates scalable linear algebra.

19. Quick Revision

Formula: $C_{ij} = \sum A_{ik} B_{kj}$. **Use:** compose learned transformations. **Cost:** $O(mnp)$. **Trap:** $*$ is not $@$. **One-liner:** matmul turns every output into a learned weighted combination.

20. Final Cheat Sheet

Definition	Input $\rightarrow$ output	Key parameter	Pros / con
Matrix product	$(m,n) @ (n,p) \rightarrow (m,p)$	inner dimension n	expressive, but costly when dense

Dot Product

1. Overview

The dot product reduces two equal-length vectors to one similarity/weighted-sum score. It is central to linear models, cosine similarity, attention, and embeddings.

2. Intuition

Two arrows pointing the same way have a large positive dot product; perpendicular arrows score zero; opposite arrows score negative.

3. Prerequisites

Vectors, multiplication, summation, and optionally angles.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Algebraic form	$x \cdot y = \sum x_i y_i$	Weighted sum in linear regression.
Geometric form	$\hat{x} \cdot \hat{y} =$	
Orthogonality	Dot product zero	Basis vectors and projections.
Cosine similarity	$\frac{x \cdot y}{\ x\ \ y\ }$	

5. Algorithm / Working Process

Multiply entries in corresponding positions, then sum the products. For a model, multiply each feature by its learned weight and add them.

6. Mathematical Foundation

$z = w^T x + b$ is a linear-model logit. For unit vectors, dot product equals cosine similarity. Softmax attention scores tokens as $QK^T / \sqrt{d_k}$.

7. Practical Implementation

```
import numpy as np

query = np.array([1., 2., 2.])
item = np.array([2., 1., 2.])
score = query @ item
cosine = score / (np.linalg.norm(query) * np.linalg.norm(item))
print(score, cosine)
```

8. Code Explanation

`query @ item` is the sum of pairwise products. Dividing by both lengths removes magnitude, leaving directional similarity.

9. Training / Evaluation

Embedding models train dot products so relevant pairs score higher than irrelevant pairs, often with contrastive, ranking, or cross-entropy loss. Evaluate retrieval with Recall@K, MRR, or nDCG.

10. Complexity and Cost

One d -dimensional dot product costs $O(d)$ time and $O(1)$ extra memory. Comparing one query to N dense embeddings costs $O(Nd)$ before indexing.

11. Common Use Cases

Linear regression/logistic regression, semantic search, recommender ranking, attention, and feature importance scoring.

12. Common Mistakes

- Comparing unnormalized vectors when cosine similarity is intended.
- Calling high dot product "similar" when vector norms differ greatly.
- Dividing by zero for a zero vector.
- Mixing feature order between $\mathbf{x}$ and $\mathbf{w}$.

13. Edge Cases / Limitations

Dot product alone cannot represent nonlinear interactions. In high dimensions, naive similarity search can be expensive and similarities may concentrate.

14. Variations

Cosine similarity normalizes magnitude; bilinear score $\mathbf{x}^T \mathbf{W} \mathbf{y}$ learns interactions; scaled dot-product attention avoids overly peaked softmax. All matter in ML interviews.

15. Related Topics

Dot product is matrix multiplication at vector scale, defines projections, and is differentiated using gradients.

16. Interview Questions

1. **Define a dot product.** Sum of coordinate-wise products.
2. **When is it zero?** For orthogonal vectors (including a zero vector).
3. **Dot product vs cosine?** Cosine normalizes vector lengths.
4. **Linear-regression prediction?** $\mathbf{w}^T \mathbf{x} + b$.
5. **Why scale attention scores?** To control variance before softmax.
6. **Can dot product be negative?** Yes, for opposing directions.
7. **Complexity?** $O(d)$.
8. **Why normalize embeddings?** To emphasize direction and make inner product equal cosine.
9. **What does a weight mean?** Its contribution per unit change in its feature, holding others fixed.
10. **How handle zero cosine denominator?** Reject or define a safe convention; zero has no direction.

17. Practice Tasks

- Compute a classifier logit from features and weights.
- Rank ten document embeddings by cosine similarity.
- Verify that two constructed vectors are orthogonal.
- Compare rankings before and after L2 normalization.

18. Project Ideas

Project	Stack / data	Resume value
FAQ semantic search	sentence-transformers, NumPy, FAQ corpus	Retrieval fundamentals.

Project	Stack / data	Resume value
Movie matcher	MovieLens embeddings	Recommender ranking concept.
Attention visualizer	PyTorch, toy sentences	Transformer intuition.

19. Quick Revision

Formula: $x \cdot y = \sum x_i y_i$. **Use:** weighted score/similarity. **Metric:** cosine or ranking metrics. **Trap:** magnitude affects raw dot product. **One-liner:** a dot product asks how strongly one vector aligns with another.

20. Final Cheat Sheet

Definition	Input → output	Cost	Best use
Inner product	two d -vectors → scalar	$O(d)$	linear scores, embeddings, attention

Norms

1. Overview

A norm measures vector or matrix size. ML uses norms to measure error, normalize features, constrain parameters, and regularize models.

2. Intuition

A norm is a distance-from-zero ruler. L1 counts total absolute movement; L2 measures straight-line (Euclidean) length.

3. Prerequisites

Vectors, absolute values, squares, roots, and dot products.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
L1 norm	$\sum x_i $	
L2 norm	$\sqrt{\sum x_i^2}$	
Squared L2	$\sum x_i^2$	Smooth, convenient regression loss.
L_∞ norm	$\max_i x_i $	x_i
Frobenius norm	$\sqrt{\sum_{ij} A_{ij}^2}$	Matrix regularization/error.

5. Algorithm / Working Process

Choose a norm based on the desired penalty, calculate it from values, then use it as a distance, normalization denominator, or regularization term in the objective.

6. Mathematical Foundation

MSE: $L = (1/N) \sum ||y_i - \hat{y}_i||_2^2$. Ridge: $L_{\text{data}} + \lambda ||w||_2^2$; Lasso: $L_{\text{data}} + \lambda ||w||_1$. Normalization is $x / ||x||_2$ when $||x||_2 > 0$.

7. Practical Implementation

```
import numpy as np

x = np.array([3., -4., 0.])
l1 = np.linalg.norm(x, ord=1)
l2 = np.linalg.norm(x)                # default is L2
unit_x = x / l2 if l2 else x          # avoid zero division
print(l1, l2, unit_x)
```

8. Code Explanation

`ord=1` requests L1; the default vector norm is L2. The conditional safely leaves a zero vector unchanged.

9. Training / Evaluation

Tune regularization strength λ on validation data. Use task metrics (accuracy, F1, RMSE), not training loss alone. Too much penalty underfits; too little can overfit.

10. Complexity and Cost

Vector norms cost $O(d)$ time and constant extra memory. Norm calculations are cheap compared with neural-network training.

11. Common Use Cases

Ridge/Lasso regression, k-NN distance, gradient clipping, embedding normalization, and reconstruction error.

12. Common Mistakes

- Treating L1 and L2 as interchangeable.
- Normalizing a zero vector without a safeguard.
- Applying feature scaling using test statistics.
- Forgetting that L2 regularization convention may differ by a factor of two.

13. Edge Cases / Limitations

L2 is sensitive to outliers; L1 is non-differentiable at zero (subgradients are used). Norms summarize magnitude but ignore semantic meaning.

14. Variations

L0 "norm" counts nonzeros (not a true norm); elastic net combines L1/L2; group norms select whole feature groups. L1/L2 are essential for placements.

15. Related Topics

Norms define Euclidean distance, cosine normalization, regularization, and gradient clipping.

16. Interview Questions

1. **What is L2 norm?** Square root of summed squares.
2. **Why use squared L2 in MSE?** It penalizes large errors and is smooth.
3. **L1 vs L2 regularization?** L1 yields sparsity; L2 shrinks weights smoothly.
4. **What does L_∞ measure?** Largest absolute component.
5. **Why normalize embeddings?** To compare directions rather than scale.
6. **Can L1 have a derivative at zero?** No unique derivative; use a subgradient.
7. **What is Frobenius norm?** L2 norm of all matrix entries.
8. **Does L2 solve outliers?** No; it emphasizes them.
9. **What is gradient clipping?** Rescaling/capping a gradient norm to stabilize updates.
10. **How select λ ?** Cross-validation or validation-set search.

17. Practice Tasks

- Implement L1, L2, and L_∞ without NumPy helpers.
- Plot L1 and L2 penalties over a 2-D weight grid.
- Add L2 weight decay to a small PyTorch model.
- Compare cosine rankings with and without normalization.

18. Project Ideas

Project	Stack / data	Resume value
Ridge vs Lasso study	scikit-learn, California Housing	Regularization trade-off.
Robust regression comparison	NumPy/sklearn, outlier-injected data	Loss-selection insight.
Embedding retrieval evaluator	FAISS/NumPy, sentence pairs	Similarity engineering.

19. Quick Revision

Main formulas: $L1 = \sum |x|$, $L2 = \sqrt{\sum x^2}$. **Use:** distance and penalties. **Trap:** L2 is outlier-sensitive. **One-liner:** norms turn a vector's size into one useful scalar.

20. Final Cheat Sheet

Norm	Effect	Best use	Caveat
L1	sparsity	feature selection	non-smooth at zero
L2	smooth shrinkage	weight decay, distance	outlier-sensitive

Eigenvalues and Eigenvectors

1. Overview

For a square matrix A , an eigenvector v keeps its direction under A ; only its scale changes by eigenvalue λ : $Av = \lambda v$. They explain PCA, stability, graph ranking, and repeated transformations.

2. Intuition

Imagine stretching a rubber sheet. Most arrows rotate, but special arrows remain on the same line. Their stretch factors are eigenvalues.

3. Prerequisites

Matrix multiplication, determinants, linear independence, and polynomial roots.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Eigenpair	$Av = \lambda v, v \neq 0$	State the definition precisely.
Characteristic equation	$\det(A - \lambda I) = 0$	Finds candidate eigenvalues.
Eigenspace	All eigenvectors for one λ , plus zero	May have dimension > 1 .
Diagonalization	$A = V\Lambda V^{-1}$ when enough independent eigenvectors	Makes powers/transforms simple.
Symmetric matrix	Real eigenvalues, orthogonal eigenvectors	Covariance/PCA rely on this.

5. Algorithm / Working Process

1. Form $A - \lambda I$.
2. Solve $\det(A - \lambda I) = 0$ for λ .
3. For each eigenvalue solve $(A - \lambda I)v = 0$.
4. Normalize/arrange vectors if needed; numerical libraries do this robustly.

6. Mathematical Foundation

PCA eigendecomposes covariance $C = (1/(N-1))X_c^T X_c$. The top eigenvectors give directions of greatest variance; eigenvalues quantify retained variance. For symmetric A , $A = Q\Lambda Q^T$.

7. Practical Implementation

```
import numpy as np

A = np.array([[2., 1.], [1., 2.]]) # symmetric
values, vectors = np.linalg.eigh(A) # preferred for symmetric matrices
largest = vectors[:, np.argmax(values)]
print(values, largest, A @ largest)
```

8. Code Explanation

`eigh` exploits symmetry and returns eigenvectors as columns. `A @ largest` should equal its eigenvalue times `largest`, subject to floating-point rounding.

9. Training / Evaluation

PCA is usually fit only on centered training data, then applied to validation/test data with the same training mean/components. Evaluate explained variance and downstream task performance.

10. Complexity and Cost

Full eigendecomposition of an $n \times n$ dense matrix is roughly $O(n^3)$ time and $O(n^2)$ memory. Use truncated/randomized methods for large PCA.

11. Common Use Cases

PCA, spectral clustering, PageRank-like methods, covariance analysis, and dynamical-system stability.

12. Common Mistakes

- Calling any vector an eigenvector without checking $Av = \lambda v$.
- Using `eig` when `eigh` applies to a symmetric covariance matrix.
- Skipping centering before PCA.
- Sorting eigenvalues but not their paired eigenvectors.

13. Edge Cases / Limitations

Some real matrices have complex eigenpairs or cannot be diagonalized. Eigenvectors are sign-ambiguous, and PCA captures variance rather than label relevance.

14. Variations

SVD works for rectangular matrices and is often numerically preferred; generalized eigenproblems appear in LDA; power iteration finds the leading pair. PCA/SVD are highly important.

15. Related Topics

Rank reveals independent directions; determinants equal eigenvalue products; PCA is an orthogonal projection onto leading eigenvectors.

16. Interview Questions

1. **Define eigenvector/eigenvalue.** $Av = \lambda v$ for nonzero v .
2. **Why PCA uses eigenvectors?** They are principal variance directions of covariance.
3. **Why center before PCA?** Otherwise variance is measured around the origin, not the mean.
4. **What does a negative eigenvalue mean?** Reversal plus scaling in that direction.
5. **Eigenvalues of a triangular matrix?** Its diagonal entries.
6. **Why `eigh` for covariance?** Covariance is symmetric.
7. **Can an eigenvector be zero?** No.
8. **Are eigenvectors unique?** Scale/sign are not; repeated eigenvalues permit many bases.
9. **What does largest PCA eigenvalue mean?** Most variance along its eigenvector.

10. **Eigen vs SVD?** Eigen is square-matrix decomposition; SVD applies to any matrix.

17. Practice Tasks

- Solve eigenpairs of a 2×2 matrix by hand.
- Verify numerical eigenpairs with `np.allclose`.
- Implement PCA using covariance plus `eigh`.
- Compare PCA with and without centering.

18. Project Ideas

Project	Stack / data	Resume value
PCA face compression	sklearn, Olivetti faces	Dimensionality reduction.
Stock covariance explorer	Pandas, public price data	Eigen-risk analysis.
Spectral image segmentation	OpenCV, scikit-learn	Advanced graph intuition.

19. Quick Revision

Formula: $Av = \lambda v$. **Use:** invariant directions/PCA. **Cost:** dense $O(n^3)$. **Trap:** PCA needs centering. **One-liner:** eigenvectors are directions a transformation does not rotate.

20. Final Cheat Sheet

Definition	Input $\rightarrow$ output	Metric	Best use
Eigendecomposition	square $A \rightarrow \lambda, V$	explained variance (PCA)	covariance/PCA, stability

Linear Transformations

1. Overview

A linear transformation maps vectors while preserving addition and scalar multiplication. Matrices represent these maps, making them the language of neural layers and geometric data changes.

2. Intuition

It can rotate, stretch, shear, or project space, but cannot bend it or move the origin. A matrix is the transformation's recipe.

3. Prerequisites

Vectors, matrices, matrix multiplication, and coordinates.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Linearity	$T(ax+by)=aT(x)+bT(y)$	Distinguish linear from affine maps.
Matrix representation	$T(x)=Ax$	Columns of A are transformed basis vectors.
Affine map	$Ax+b$	Dense layers use this, not strictly linear if $b \neq 0$.
Composition	$T_2(T_1(x))=(A_2A_1)x$	Explains layer stacking/order.
Kernel/image	inputs sent to zero / reachable outputs	Connect to rank and information loss.

5. Algorithm / Working Process

Represent input in a basis, multiply by the transformation matrix, optionally add a bias, then apply a nonlinearity to make a neural network expressive.

6. Mathematical Foundation

$T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ has $A \in \mathbb{R}^{(m \times n)}$. The j th column is $T(e_j)$. A neural layer $h = \sigma(Wx+b)$ is affine followed by nonlinear σ ; without σ , stacked layers collapse to one affine map.

7. Practical Implementation

```
import numpy as np

rotation_90 = np.array([[0., -1.], [1., 0.]])
x = np.array([2., 1.])
rotated = rotation_90 @ x
affine = rotated + np.array([0.5, 0.5])
print(rotated, affine)
```

8. Code Explanation

The matrix rotates a 2-D vector 90 degrees counterclockwise. Adding $[0.5, 0.5]$ translates it, so the second operation is affine rather than linear.

9. Training / Evaluation

Models learn transformation parameters via gradients. Inspect validation loss/task metrics, use regularization, and ensure input distributions match training preprocessing.

10. Complexity and Cost

Applying $A: (m, n)$ to one vector costs $O(mn)$; a batch N costs $O(Nmn)$. Parameters consume $O(mn)$ memory.

11. Common Use Cases

Dense layers, image augmentation, PCA projection, coordinate conversion, and learned embedding maps.

12. Common Mistakes

- Calling $Ax+b$ linear instead of affine.
- Reversing composition order.
- Forgetting transforms act on the origin differently when bias exists.
- Stacking only linear layers and expecting new expressiveness.

13. Edge Cases / Limitations

Linear maps cannot model XOR or curved boundaries alone. A rank-deficient map loses information and cannot be inverted on all inputs.

14. Variations

Orthogonal transforms preserve L2 norm; diagonal transforms scale axes; convolution is a structured linear map; nonlinear activations create deep networks. Essential for interviews.

15. Related Topics

Matrices encode transforms; rank measures output dimension; eigenvectors reveal invariant directions; Jacobians locally linearize nonlinear functions.

16. Interview Questions

1. **Linearity conditions?** Preserve addition and scalar multiplication.
2. **Is $Ax+b$ linear?** Only if $b=0$; otherwise affine.
3. **What does each matrix column represent?** Image of a basis vector.
4. **Why activations in neural nets?** They prevent many layers from collapsing into one linear map.
5. **Composition order?** A_2A_1x applies A_1 first.
6. **What is kernel?** Inputs mapped to zero.
7. **What is image/range?** Set of attainable outputs.
8. **When invertible?** Square, full-rank transformation.
9. **What preserves Euclidean length?** An orthogonal matrix.
10. **How is convolution linear?** It distributes over sums and scalar multiples before activation.

17. Practice Tasks

- Construct matrices for scaling, reflection, and rotation.
- Demonstrate that composition order changes a point.
- Build a NumPy linear layer with a ReLU.
- Show why a two-layer linear network collapses algebraically.

18. Project Ideas

Project	Stack / data	Resume value
2-D transformation playground	NumPy, Matplotlib	Clear geometry communication.

Project	Stack / data	Resume value
MLP from scratch	NumPy, MNIST subset	Neural-network foundations.
Image augmentation lab	OpenCV, CIFAR-10	Vision preprocessing knowledge.

19. Quick Revision

Key idea: $T(x)=Ax$ preserves straight-line structure. **Use:** learned layers and geometry. **Trap:** bias makes it affine. **One-liner:** matrices are reusable machines for linear transformations.

20. Final Cheat Sheet

Definition	Input → output	Cost	Best use
Linear map	$x \in \mathbb{R}^n \rightarrow Ax \in \mathbb{R}^m$	$O(mn)$	layers, projections, coordinate changes

Rank, Determinant, and Inverse

1. Overview

Rank counts independent information in a matrix, determinant measures square-map volume scaling, and an inverse undoes an invertible square transformation. They diagnose solvability, redundancy, and numerical stability.

2. Intuition

If matrix columns point in fewer directions than advertised, rank is low. Determinant zero means a shape was flattened. An inverse is the reverse machine—possible only when no direction was flattened.

3. Prerequisites

Matrices, elimination, linear independence, and multiplication.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Rank	number of independent rows/columns	Detects redundant features.
Determinant	signed volume scale of square A	$\det(A)=0$ iff singular.
Inverse	$AA^{-1}=A^{-1}A=I$	Exists iff square full rank.
Singularity	non-invertible / rank deficient	Multicollinearity causes it.
Condition number	sensitivity to perturbations	Near-singular matrices are unstable.

5. Algorithm / Working Process

Use Gaussian elimination/SVD to estimate rank. For a square matrix, check conditioning before solving a system; solve $Ax=b$ directly rather than explicitly forming A^{-1} .

6. Mathematical Foundation

For square A , $\det(A) \neq 0 \Leftrightarrow \text{rank}(A)=n \Leftrightarrow A^{-1}$ exists. In 2-D, $\det(\begin{bmatrix} a & b \\ c & d \end{bmatrix})=ad-bc$; if nonzero, inverse is $(1/\det)\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$.

7. Practical Implementation

```
import numpy as np

A = np.array([[2., 1.], [1., 1.]])
print("rank:", np.linalg.matrix_rank(A), "det:", np.linalg.det(A))
inv_A = np.linalg.inv(A)
print(np.allclose(A @ inv_A, np.eye(2)))
```

8. Code Explanation

`matrix_rank` uses a numerical tolerance. `allclose` checks the inverse identity despite floating-point rounding.

9. Training / Evaluation

For linear regression, check collinearity and scale features. Prefer regularization or SVD-based solvers for ill-conditioned data. Evaluate predictive metrics on held-out data; an invertible design matrix is not a quality guarantee.

10. Complexity and Cost

Determinant, inverse, and dense linear solves are about $O(n^3)$ time, with $O(n^2)$ memory. SVD is more expensive but robust.

11. Common Use Cases

Feature-redundancy checks, least squares, normalizing flows (log determinant), geometric transforms, and covariance diagnostics.

12. Common Mistakes

- Computing `inv(A) @ b` instead of `solve(A,b)`.
- Equating a tiny nonzero determinant with numerical safety.
- Expecting a non-square matrix to have a two-sided inverse.
- Ignoring duplicate/highly correlated features.

13. Edge Cases / Limitations

Floating-point rank is tolerance-dependent. Determinants overflow/underflow for large matrices. Inverses magnify noise when conditioning is poor.

14. Variations

Left/right pseudoinverse for rectangular or rank-deficient matrices; LU/QR/Cholesky factorizations solve systems efficiently. Pseudoinverse and conditioning are placement-important.

15. Related Topics

Rank determines solution structure; determinant is eigenvalue product; inverse is used by projections and some optimization formulas.

16. Interview Questions

1. **What is rank?** Number of independent row/column directions.
2. **When does inverse exist?** For a square full-rank matrix.
3. **What does determinant zero mean?** Singular transformation; volume collapses.
4. **Why not compute inverses for solves?** It is slower and less numerically stable.
5. **Rank of outer product uv^T ?** At most one (zero if either vector is zero).
6. **What is multicollinearity?** Features are highly linearly dependent.
7. **What handles singular least squares?** Pseudoinverse, QR/SVD, or regularization.
8. **Does nonzero determinant guarantee stable inverse?** No; it may be nearly singular.
9. **What is condition number?** A measure of solution sensitivity to small perturbations.
10. **Rank row vs column?** Always equal.

17. Practice Tasks

- Create a rank-1 matrix and verify its rank.
- Compare `solve` with `inv @ b` on a poorly conditioned matrix.
- Detect collinear columns in a dataset.
- Compute a pseudoinverse least-squares solution.

18. Project Ideas

Project	Stack / data	Resume value
Multicollinearity report	Pandas/sklearn, housing data	Practical regression diagnostics.
Stable linear solver demo	NumPy, synthetic systems	Numerical-method awareness.
Flow log-det notebook	PyTorch, toy density data	Generative-model connection.

19. Quick Revision

Facts: full-rank square $\Leftrightarrow$ nonzero determinant $\Leftrightarrow$ inverse. **Use:** solvability/diagnostics. **Trap:** never default to explicit inverse. **One-liner:** rank says how much information survives a matrix.

20. Final Cheat Sheet

Concept	Test	Use	Caveat
Rank / det / inverse	$\text{rank}=n, \text{det}\neq 0$	solve and diagnose maps	conditioning matters

Projections

1. Overview

A projection maps a vector to a subspace, retaining the component explained by that subspace. It underlies least squares, PCA, embeddings, and residual analysis.

2. Intuition

It is a vector's shadow on a line or plane. The shadow is the closest point in that subspace under Euclidean distance.

3. Prerequisites

Dot products, norms, orthogonality, matrices, and least squares.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Scalar projection	signed length $\mathbf{x} \cdot \mathbf{u}$ for unit $\mathbf{u}$	Measures alignment.
Vector projection	$\text{proj}_{\mathbf{u}}(\mathbf{x}) = (\mathbf{x} \cdot \mathbf{u}) / (\mathbf{u} \cdot \mathbf{u}) \mathbf{u}$	Closest point on a line.
Orthogonal residual	$\mathbf{r} = \mathbf{x} - \text{proj}(\mathbf{x})$ and $\mathbf{r} \perp \mathbf{u}$	Least-squares condition.
Projection matrix	$\mathbf{P} = \mathbf{U} \mathbf{U}^T$ for orthonormal $\mathbf{U}$	Applies projection to any vector.
Idempotence	$\mathbf{P}^2 = \mathbf{P}$	Defining property to recognize.

5. Algorithm / Working Process

Choose an orthonormal basis $\mathbf{U}$ for the target subspace, compute $\mathbf{z} = \mathbf{U}^T \mathbf{x}$, reconstruct $\mathbf{x}_{\text{hat}} = \mathbf{U} \mathbf{z}$, and inspect residual $\mathbf{x} - \mathbf{x}_{\text{hat}}$.

6. Mathematical Foundation

For full-column-rank $\mathbf{A}$, projection onto $\text{col}(\mathbf{A})$ is $\mathbf{P} = \mathbf{A}(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$; use QR/SVD numerically instead. $\mathbf{P}$ is symmetric and idempotent for an orthogonal projection.

7. Practical Implementation

```
import numpy as np

x = np.array([3., 1.])
u = np.array([1., 1.])
```

```
projection = (x @ u) / (u @ u) * u
residual = x - projection
print(projection, residual, residual @ u) # last value is 0
```

8. Code Explanation

The scalar ratio measures how much of u is present in x . Subtracting the shadow leaves an orthogonal residual.

9. Training / Evaluation

Fit PCA/projector parameters on training data only. Evaluate reconstruction error $\|x - \hat{x}\|^2$, explained variance, or downstream accuracy. Retaining too few components underfits structure.

10. Complexity and Cost

Projecting onto k orthonormal vectors costs $O(dk)$ per vector and stores $O(dk)$ components. Building a full $d \times d$ projector is often unnecessary.

11. Common Use Cases

PCA compression, least-squares fitted values, removing nuisance directions, and embedding dimensionality reduction.

12. Common Mistakes

- Using $x \cdot u$ as vector projection when u is not unit length.
- Assuming every projection is orthogonal.
- Constructing P explicitly for high-dimensional data.
- Fitting PCA on test data.

13. Edge Cases / Limitations

Projection loses discarded information. Euclidean projection may not match semantic similarity, and non-orthogonal bases can be ill-conditioned.

14. Variations

Orthogonal vs oblique projections; PCA projection; random projection for scalable dimension reduction; kernel projection for nonlinear structure. PCA is vital for placements.

15. Related Topics

Dot products calculate shadows; norms measure reconstruction error; least squares is projection onto a column space; eigenvectors define PCA subspaces.

16. Interview Questions

1. **What is a projection?** A mapping to a subspace, often the nearest point there.

2. **Projection onto vector u ?** $(x \cdot u) / (u \cdot u)u$.
3. **Why is residual orthogonal?** At the closest point, moving along the subspace cannot reduce error.
4. **Projection-matrix properties?** $P^2=P$; orthogonal P also has $P^T=P$.
5. **Why PCA is projection?** It maps data onto selected principal directions.
6. **What does $U^T x$ produce?** Coordinates in the orthonormal basis.
7. **Does projection preserve length?** Only if vector already lies in the subspace.
8. **How avoid inverse in P ?** QR or SVD.
9. **What is reconstruction error?** Norm of original minus projection.
10. **Why use random projections?** Faster reduction with approximate distance preservation.

17. Practice Tasks

- Project several points onto a line and plot residuals.
- Verify $P@P=P$ for an orthonormal basis.
- Implement PCA reconstruction at different component counts.
- Compare QR-based and normal-equation projections.

18. Project Ideas

Project	Stack / data	Resume value
PCA image compressor	sklearn, Fashion-MNIST	Dimensionality reduction metrics.
Least-squares visualizer	NumPy, synthetic data	Explains regression geometry.
Random-projection search	sklearn, text embeddings	Scalability perspective.

19. Quick Revision

Formula: $\text{proj}_u x = (x \cdot u) / (u \cdot u)u$. **Use:** best subspace approximation. **Metric:** reconstruction error. **Trap:** normalize correctly. **One-liner:** projection keeps the part of a vector a chosen subspace can explain.

20. Final Cheat Sheet

Definition	Input $\rightarrow$ output	Cost	Best use
Orthogonal projection	$x \rightarrow UU^T x$	$O(dk)$	PCA, least squares, compression

Systems of Linear Equations

1. Overview

A system $Ax=b$ asks for parameter vector x that satisfies several linear constraints. Regression fitting, calibration, and optimization all reduce to such systems.

2. Intuition

Each equation is a line/plane; solving finds their shared intersection. With noisy data, exact intersection may not exist, so least squares finds the closest fit.

3. Prerequisites

Matrices, rank, inverse, projections, and basic algebra.

4. Core Concepts

Subtopic	Meaning and example	Why / interview angle
Unique solution	square A full rank	One intersection.
No solution	inconsistent constraints	Typical noisy overdetermined data.
Infinite solutions	free variables	Rank deficient/underdetermined.
Gaussian elimination	row-reduce augmented matrix	Fundamental solving method.
Least squares	minimize `	

5. Algorithm / Working Process

1. Form **A** and **b**.
2. Check dimensions/rank.
3. For square well-conditioned **A**, use a solver.
4. For noisy/rectangular **A**, use least squares (QR/SVD).
5. Inspect residuals and validate predictive performance.

6. Mathematical Foundation

Least squares minimizes $\|Ax - b\|_2^2$; setting gradient to zero yields normal equations $A^T A x = A^T b$. Do not explicitly invert $A^T A$; $x = A^+ b$ is the pseudoinverse form.

7. Practical Implementation

```
import numpy as np

A = np.array([[1., 1.], [1., 2.], [1., 3.]]) # intercept + feature
b = np.array([2., 2.9, 4.2])
x, residuals, rank, _ = np.linalg.lstsq(A, b, rcond=None)
print("[intercept, slope] =", x, "rank =", rank)
```

8. Code Explanation

`lstsq` solves an overdetermined system stably. Here it fits the best straight line rather than demanding every noisy point lie exactly on it.

9. Training / Evaluation

Rows are samples and columns are predictors. Split before scaling/feature engineering, fit on train, then report MAE/RMSE/ R^2 on validation/test. Cross-validate regularization for collinearity.

10. Complexity and Cost

Dense square solving is $O(n^3)$. Least squares via QR is roughly $O(Nd^2)$ when $N \geq d$; SVD costs more but handles rank issues robustly.

11. Common Use Cases

Linear regression, sensor calibration, triangulation, portfolio constraints, and fitting local linear models.

12. Common Mistakes

- Solving noisy overdetermined data as if an exact solution exists.
- Using normal-equation inverse blindly.
- Ignoring scaling and multicollinearity.
- Evaluating regression only on training residuals.

13. Edge Cases / Limitations

Linear assumptions fail for nonlinear relationships; outliers distort least squares; underdetermined systems need a selection rule such as minimum-norm solution or regularization.

14. Variations

Weighted least squares handles unequal noise; ridge adds λI ; robust regression changes loss; iterative solvers handle huge sparse systems. Ridge/least squares are essential.

15. Related Topics

Rank classifies solution counts; projections explain least squares; derivatives derive normal equations; inverse/pseudoinverse provide solution forms.

16. Interview Questions

1. **When unique solution?** A square and full rank.
2. **Overdetermined means?** More equations than unknowns.
3. **How solve noisy regression?** Least squares.
4. **Normal equations?** $A^T A x = A^T b$.
5. **Why avoid inverse?** Numerical stability and cost.
6. **What is residual?** $b - Ax$.
7. **Why residual is orthogonal in least squares?** It is orthogonal to $\text{col}(A)$.
8. **What handles singular $A^T A$?** SVD/pseudoinverse or ridge.
9. **What is underdetermined?** Fewer independent equations than unknowns.
10. **Why standardize regression features?** Better conditioning and comparable regularization.

17. Practice Tasks

- Solve a 2×2 system by elimination.
- Fit a line using `lstsq` and plot residuals.
- Add duplicated columns and observe rank deficiency.

- Compare ordinary and ridge regression under collinearity.

18. Project Ideas

Project	Stack / data	Resume value
House-price baseline	sklearn, Ames Housing	Strong regression baseline.
Sensor calibrator	NumPy, generated measurements	Solving noisy systems.
Regularized demand model	Pandas/sklearn, retail data	Business ML application.

19. Quick Revision

Goal: solve $Ax=b$; noisy case minimizes $\|Ax-b\|^2$. **Use:** regression. **Metric:** RMSE/MAE. **Trap:** inverse normal equations. **One-liner:** least squares finds the closest attainable output to b .

20. Final Cheat Sheet

Situation	Method	Cost	Caveat
square/full rank	<code>solve(A,b)</code>	$O(n^3)$	conditioning
noisy/rectangular	<code>lstsq(A,b)</code>	$\sim O(Nd^2)$	outliers

Derivatives

1. Overview

A derivative measures instantaneous output change caused by a tiny input change. Optimizers use derivatives to reduce ML losses.

2. Intuition

It is the slope of a curve at one point: positive means increasing, negative decreasing, and zero may be a minimum, maximum, or saddle.

3. Prerequisites

Functions, algebra, coordinate graphs, and limits intuition.

4. Core Concepts

Subtopic	Meaning	Interview angle
Definition	$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)]/h$	Formal local rate of change.
Rules	power, sum, product, quotient	Differentiate losses.
Critical point	$f'(x) = 0$ or undefined	Candidate extrema only.
Finite difference	approximate slope from nearby values	Gradient-checking baseline.

5. Algorithm / Working Process

Define $L(\theta)$, compute $dL/d\theta$, update $\theta \leftarrow \theta - \eta(dL/d\theta)$, and select settings using validation performance.

6. Mathematical Foundation

$d(x^n)/dx = nx^{(n-1)}$ and $(fg)' = f'g + fg'$. Gradient descent moves opposite the derivative because that locally lowers loss.

7. Practical Implementation

```
def loss(w): return (w - 3.0) ** 2
w, lr = 0.0, 0.1
for _ in range(20):
    w -= lr * 2 * (w - 3.0)
print(w, loss(w))
```

8. Code Explanation

The loss has minimum at $w=3$; $2(w-3)$ gives its slope and shrinks each update near the minimum.

9. Training / Evaluation

Update on training loss but pick epochs/hyperparameters from validation loss. Too large a learning rate diverges; too small is slow.

10. Complexity and Cost

One scalar derivative is $O(1)$. Autodiff computes a network's derivatives at roughly a small multiple of forward-pass cost.

11. Common Use Cases

Gradient descent, curve fitting, backpropagation, and sensitivity analysis.

12. Common Mistakes

- Confusing derivative with function value.
- Calling every zero derivative a minimum.
- Choosing learning rate without checking loss curves.
- Using unstable finite-difference step sizes.

13. Edge Cases / Limitations

Non-differentiable points need subgradients; flat or ill-conditioned surfaces slow training.

14. Variations

Central differences improve numerical checking; autodiff applies exact symbolic rules numerically; subgradients cover ReLU/L1. These are placement-important.

15. Related Topics

Partial derivatives handle many parameters; chain rule enables backprop; Hessian measures curvature.

16. Interview Questions

1. **What is a derivative?** Local output rate of change.
2. **Why negative slope in descent?** It is locally downhill.
3. **Derivative of x^2 ?** $2x$.
4. **Does zero derivative ensure a minimum?** No.
5. **What controls update size?** Learning rate.
6. **Why use autodiff?** Accurate, efficient repeated chain-rule application.
7. **What is finite difference?** A nearby-value slope approximation.
8. **Derivative of ReLU at zero?** A chosen subgradient, commonly 0.
9. **Why clip gradients?** Prevent unstable updates.
10. **What indicates divergence?** Growing/NaN loss or parameters.

17. Practice Tasks

- Differentiate polynomials and check them numerically.
- Optimize a quadratic with three learning rates.
- Implement a finite-difference gradient checker.
- Find a stationary point that is not a minimum.

18. Project Ideas

Project	Stack / data	Resume value
Gradient-descent visualizer	NumPy, Matplotlib	Optimization communication.
Linear regression from scratch	NumPy, housing data	Loss/derivative fluency.
Autodiff checker	PyTorch, toy functions	Debugging maturity.

19. Quick Revision

Formula: $f'(x) = \lim \Delta f / \Delta x$. **Use:** optimize loss. **Trap:** stationary does not mean minimum. **One-liner:** derivatives tell an optimizer which way is uphill.

20. Final Cheat Sheet

Definition	Input → output	Key hyperparameter	Best use
Derivative	scalar → scalar slope	learning rate	gradient descent

Partial Derivatives

1. Overview

A partial derivative changes one input while holding all others fixed. It gives each parameter's local effect on a multivariable loss.

2. Intuition

On a mountain surface, it is one coordinate-direction slope while refusing to move along the other axes.

3. Prerequisites

Single-variable derivatives, multivariable functions, and vectors.

4. Core Concepts

Subtopic	Meaning	Interview angle
$\partial f / \partial x_i$	coordinate-specific slope	A gradient component.
Holding fixed	other variables are constants	Avoid wrong derivatives.
Mixed partial	$\partial^2 f / (\partial x \partial y)$	Captures interaction/curvature.
Total derivative	includes indirect dependence	Contrast with partial.

5. Algorithm / Working Process

Differentiate the loss with respect to each parameter while freezing the others, collect the resulting values, then update all parameters.

6. Mathematical Foundation

For $f(x, y) = x^2y + 3y$, $\partial f / \partial x = 2xy$ and $\partial f / \partial y = x^2 + 3$. Under smoothness conditions, mixed partial order can be swapped.

7. Practical Implementation

```
import torch
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
(x**2 * y + 3*y).backward()
print(x.grad.item(), y.grad.item()) # 12.0, 7.0
```

8. Code Explanation

`backward()` computes one partial for each leaf tensor marked `requires_grad=True`.

9. Training / Evaluation

Monitor gradient norms and validation metrics. Clear accumulated framework gradients before the next batch; scales affect their magnitude.

10. Complexity and Cost

Finite differences cost one evaluation per parameter; reverse-mode autodiff efficiently gives all partials of one scalar loss.

11. Common Use Cases

Parameter learning, saliency, sensitivity analysis, and constrained optimization.

12. Common Mistakes

- Not holding other variables constant.
- Forgetting dependency requiring chain rule.
- Accumulating PyTorch gradients without zeroing.
- Comparing gradients across unscaled features.

13. Edge Cases / Limitations

Partial derivatives depend on scaling and parameterization; non-smooth points need subgradients; correlated inputs can make individual effects misleading.

14. Variations

Directional derivative, total derivative, and mixed partial. All are useful calculus interview extensions.

15. Related Topics

Gradients stack partials; Jacobians stack them for vector outputs; chain rule propagates them.

16. Interview Questions

1. **Define a partial derivative.** Slope in one variable holding others fixed.
2. $\partial(x^2y)/\partial x$? $2xy$.
3. $\partial(x^2y)/\partial y$? x^2 .
4. **Why needed in ML?** Loss has many parameters.
5. **What is mixed partial?** Differentiate with respect to two variables.
6. **When do mixed partials agree?** With sufficient smoothness.
7. **How retrieve in PyTorch?** `.backward()` then `.grad`.
8. **Why zero gradients?** They accumulate by default.
9. **Partial vs total?** Total includes indirect paths.
10. **What affects magnitude?** Feature/loss scale.

17. Practice Tasks

- Derive partials of a polynomial.
- Verify them with PyTorch.
- Plot coordinate slopes on a surface.

- Debug gradient accumulation.

18. Project Ideas

Project	Stack / data	Resume value
Loss-landscape explorer	PyTorch, synthetic data	Optimization intuition.
Feature-sensitivity report	sklearn/PyTorch, tabular data	Explainability basics.
Gradient-check utility	NumPy, custom layer	ML debugging skill.

19. Quick Revision

Key idea: one-coordinate slope. **Formula:** $\partial(x^2y)/\partial x = 2xy$. **Trap:** hold others constant. **One-liner:** partials are per-parameter learning signals.

20. Final Cheat Sheet

Definition	Input → output	Best use	Caveat
Partial derivative	multivariable scalar → scalar	parameter sensitivity	scale-dependent

Chain Rule

1. Overview

The chain rule differentiates composed functions. It is the mathematical engine of backpropagation through neural-network layers.

2. Intuition

If a weight changes an activation and that activation changes loss, multiply those two effects to get the weight's effect on loss.

3. Prerequisites

Derivatives, partial derivatives, function composition, and multiplication.

4. Core Concepts

Subtopic	Meaning	Interview angle
Scalar rule	$d f(g(x))/dx = f'(g(x))g'(x)$	Core formula.
Computational graph	operations as connected nodes	Backprop implementation.
Upstream gradient	$\partial L / \partial \text{output}$ arriving at an operation	Multiply by local derivative.
Reverse mode	propagate loss-to-input	Efficient for one loss, many weights.

5. Algorithm / Working Process

Forward-pass/cache activations; set $\partial L / \partial L = 1$; traverse operations backward; multiply local and upstream derivatives; add contributions at branches; update parameters.

6. Mathematical Foundation

For $z=wx+b$, $a=\sigma(z)$, $L=L(a)$: $\partial L / \partial w = (\partial L / \partial a) \sigma'(z)x$. For sigmoid, $\sigma'(z) = \sigma(z)(1-\sigma(z))$.

7. Practical Implementation

```
import torch
w = torch.tensor(0.5, requires_grad=True)
pred = torch.sigmoid(w * 2.0)
loss = (pred - 1.0) ** 2
loss.backward()
print(w.grad.item())
```

8. Code Explanation

Autograd follows $w \rightarrow \text{multiply} \rightarrow \text{sigmoid} \rightarrow \text{loss}$ backward, multiplying each local derivative.

9. Training / Evaluation

Check that every trainable tensor receives a gradient, track gradient norms and validation loss, and guard against vanishing/exploding gradients.

10. Complexity and Cost

Reverse mode costs roughly a small multiple of forward compute and stores activations, creating a memory–compute trade-off.

11. Common Use Cases

Backpropagation, differentiable losses, neural rendering, policy gradients, and differentiable programming.

12. Common Mistakes

- Adding rather than multiplying along a chain.
- Forgetting paths add at a branch.
- Detaching tensors/in-place mutations.
- Ignoring saturated activation derivatives.

13. Edge Cases / Limitations

Long products can vanish/explode. Discrete or non-differentiable operations need subgradients, estimators, or relaxation.

14. Variations

Forward-mode is useful for few inputs/many outputs; reverse-mode is standard backprop; checkpointing trades compute for activation memory.

15. Related Topics

Partial derivatives are local pieces, gradients collect parameter effects, and Jacobians compose via a matrix chain rule.

16. Interview Questions

1. **State the chain rule.** Outer derivative at inner times inner derivative.
2. **Why backward?** Reverse mode handles one loss and many weights efficiently.
3. **Upstream gradient?** Loss derivative w.r.t. an operation's output.
4. **Why cache activations?** Local derivatives need forward values.
5. **Why vanish?** Repeated factors below one shrink products.
6. **Why explode?** Repeated large factors grow products.
7. **At branches?** Sum downstream contributions.
8. **Sigmoid derivative?** $\sigma(1-\sigma)$.
9. **How save memory?** Activation checkpointing.
10. **Why ReLU helped historically?** Less saturation on positive values.

17. Practice Tasks

- Derive one-neuron MSE gradient.
- Draw its computational graph.
- Compare manual gradient to PyTorch.
- Diagnose an accidental `.detach()`.

18. Project Ideas

Project	Stack / data	Resume value
Micrograd-style autodiff	Python/NumPy	Backprop depth.
MLP from scratch	NumPy, spiral data	Manual gradients.
Gradient-flow study	PyTorch, MNIST	Architecture diagnostics.

19. Quick Revision

Formula: $d f(g(x))/dx = f'(g(x))g'(x)$. **Trap:** branches add, chains multiply. **One-liner:** chain rule credits a parameter through every downstream operation.

20. Final Cheat Sheet

Definition	Flow	Cost	Best use
Chain rule	local gradients backward	activation memory	neural training

Gradients

1. Overview

The gradient is the vector of partial derivatives of a scalar function. It points toward steepest local increase; its negative drives first-order optimization.

2. Intuition

On a loss landscape, the gradient is the arrow pointing most uphill. Gradient descent walks the opposite direction.

3. Prerequisites

Vectors, norms, partial derivatives, and chain rule.

4. Core Concepts

Subtopic	Meaning	Interview angle
Gradient	$\nabla f = [\partial f / \partial x_1, \dots, \partial f / \partial x_d]$	Shape matches parameters.
Directional derivative	$\nabla f \cdot \mathbf{u}$ for unit $\mathbf{u}$	Gradient is steepest ascent.
Stationary point	$\nabla f = \mathbf{0}$	Could be min/max/saddle.
Stochastic gradient	batch estimate of full gradient	Scales deep learning.

5. Algorithm / Working Process

Sample a batch, forward-pass, compute loss, backpropagate gradient, optionally clip/regularize it, optimizer-step, and validate periodically.

6. Mathematical Foundation

For MSE linear regression, $L = (1/N) \|X\mathbf{w} - \mathbf{y}\|^2$, $\nabla_{\mathbf{w}} L = (2/N) X^T (X\mathbf{w} - \mathbf{y})$. Update $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$; Adam adapts/scales gradient moments.

7. Practical Implementation

```
import torch
w = torch.tensor(0.0, requires_grad=True)
opt = torch.optim.SGD([w], lr=0.1)
for _ in range(20):
    opt.zero_grad(); loss = (w - 3.0).pow(2)
    loss.backward(); opt.step()
print(w.item())
```

8. Code Explanation

`backward()` fills `w.grad`; `zero_grad()` prevents accumulation; `step()` applies negative-gradient SGD.

9. Training / Evaluation

Use minibatches, train/validation split, task metrics, learning-rate schedules, and early stopping. Diagnose overfitting from diverging train/validation performance.

10. Complexity and Cost

One full gradient costs one pass over all data; minibatches cut per-step cost but add noise. GPU memory is dominated by activations and parameters.

11. Common Use Cases

Training neural networks, logistic/linear regression, adversarial examples, and gradient-based explainability.

12. Common Mistakes

- Forgetting to zero gradients.
- Using train metrics as final evaluation.
- Learning rate too large/small.
- Ignoring input/target scaling and exploding gradients.

13. Edge Cases / Limitations

Nonconvex objectives have saddles/local minima; stochastic gradients are noisy; gradients can vanish/explode in deep/recurrent networks.

14. Variations

Batch, SGD, minibatch, momentum, RMSProp, Adam, and gradient clipping. SGD/Adam trade-offs are a standard placement question.

15. Related Topics

Partials form gradients; chain rule calculates them; Hessian adds curvature; Jacobian generalizes to vector outputs.

16. Interview Questions

1. **What is a gradient?** Vector of scalar-loss partial derivatives.
2. **Why negative gradient?** Locally steepest decrease.
3. **Full batch vs SGD?** Exact/expensive versus noisy/cheap updates.
4. **Why minibatches?** Hardware efficiency plus manageable noise.
5. **What is gradient clipping?** Capping/rescaling large norms.
6. **Why zero gradients?** Frameworks accumulate them.
7. **What is a saddle?** Stationary point with both up/down directions.
8. **Why normalize features?** Better conditioning and stable updates.
9. **What does momentum do?** Smooths and accelerates persistent directions.
10. **Adam drawback?** More hyperparameters and sometimes worse generalization than tuned SGD.

17. Practice Tasks

- Derive linear-regression gradient.
- Train it from scratch with minibatches.
- Compare SGD, momentum, and Adam.
- Plot gradient norms to detect explosion.

18. Project Ideas

Project	Stack / data	Resume value
Optimizer benchmark	PyTorch, Fashion-MNIST	Experimental rigor.
Regression from scratch	NumPy, housing data	Math-to-code skill.
Gradient-based saliency	PyTorch, CIFAR-10	Explainable vision.

19. Quick Revision

Formula: $\nabla f = [\text{partials}]$; update $\theta - \eta \nabla L$. **Use:** training. **Metric:** validation task score. **Trap:** gradient zero may be saddle. **One-liner:** the gradient is the loss surface's local compass.

20. Final Cheat Sheet

Definition	Input → output	Hyperparameters	Best use
Gradient	scalar loss → parameter vector	LR, batch size, optimizer	first-order learning

Jacobian

1. Overview

The Jacobian collects first partial derivatives of a vector-valued function. It is the local linear map behind neural layers, coordinate changes, and normalizing flows.

2. Intuition

Near one input, a nonlinear function behaves like a matrix. The Jacobian is that best local matrix approximation.

3. Prerequisites

Matrices, partial derivatives, gradients, and chain rule.

4. Core Concepts

Subtopic	Meaning	Interview angle
Definition	$J_{ij} = \partial f_i / \partial x_j$	Shape is outputs × inputs.
Local linearization	$f(x + \Delta x) \approx f(x) + J \Delta x$	Why Jacobian matters.

Subtopic	Meaning	Interview angle
Jacobian-vector product	Jv without building J	Efficient autodiff primitive.
Determinant	volume scale when J is square	Change of variables/flows.

5. Algorithm / Working Process

For every output component, differentiate it with respect to every input component; arrange rows by output. In practice request JVP/VJP from autodiff instead of materializing large Jacobians.

6. Mathematical Foundation

If $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, then $J \in \mathbb{R}^{(m \times n)}$. For composition $h=f \circ g$, $J_h(x) = J_f(g(x))J_g(x)$. For scalar output, the Jacobian is the gradient row (convention varies).

7. Practical Implementation

```
import torch

def f(x): return torch.stack([x[0]**2 + x[1], x[0] * x[1]])
x = torch.tensor([2.0, 3.0])
J = torch.autograd.functional.jacobian(f, x)
print(J) # [[4, 1], [3, 2]]
```

8. Code Explanation

The first row differentiates first output w.r.t. both inputs; the second row does the same for second output.

9. Training / Evaluation

Most training only needs vector-Jacobian products from a scalar loss. Explicit Jacobians matter for flow likelihoods, sensitivity, and stability; validate against finite differences on small inputs.

10. Complexity and Cost

A dense $m \times n$ Jacobian needs $O(mn)$ memory and can be costly to form. JVP/VJP are much cheaper when only a product is needed.

11. Common Use Cases

Backprop local derivatives, normalizing flows, robotics/control, sensitivity analysis, and nonlinear least squares.

12. Common Mistakes

- Transposing output/input axes.
- Confusing a gradient with a general Jacobian.
- Materializing a giant Jacobian when VJP suffices.
- Ignoring determinant sign/absolute value in density transforms.

13. Edge Cases / Limitations

Jacobians may be undefined at kinks and very ill-conditioned. Explicit storage is impractical for high-dimensional image/LLM outputs.

14. Variations

JVP (forward mode), VJP (reverse mode), block Jacobians, and log-absolute-determinant Jacobians in flows. JVP/VJP distinction is advanced interview material.

15. Related Topics

Gradient is scalar-output Jacobian; chain rule composes Jacobians; Hessian differentiates a gradient again.

16. Interview Questions

1. **What is Jacobian?** Matrix of first partials for vector output.
2. **Its shape for $\mathbb{R}^n \rightarrow \mathbb{R}^m$?** $m \times n$.
3. **Gradient vs Jacobian?** Gradient is for scalar output; Jacobian generalizes it.
4. **Why local linearization?** It approximates nonlinear behavior nearby.
5. **Composition rule?** Multiply Jacobians in function order.
6. **What is JVP?** Product Jv .
7. **What is VJP?** Product $v^T J$, used by backprop.
8. **Why avoid full J?** Memory/time cost.
9. **Where determinant used?** Change-of-variable density formulas.
10. **How validate a Jacobian?** Finite differences on small cases.

17. Practice Tasks

- Derive Jacobian of a two-output function.
- Confirm it with PyTorch.
- Compute a JVP using finite difference.
- Compare full Jacobian memory with VJP.

18. Project Ideas

Project	Stack / data	Resume value
Neural sensitivity dashboard	PyTorch, tabular data	Local explainability.
Planar normalizing flow	PyTorch, 2-D samples	Generative-AI math.
Robot kinematics demo	NumPy, arm geometry	Applied Jacobians.

19. Quick Revision

Formula: $J_{ij} = \partial f_i / \partial x_j$. **Use:** local linear map. **Cost:** full $O(mn)$ memory. **Trap:** axis order. **One-liner:** a Jacobian is the matrix version of slope.

20. Final Cheat Sheet

Definition	Input → output	Best practice	Best use
Jacobian	$\mathbb{R}^n \rightarrow \mathbb{R}^m \rightarrow m \times n$ partials	use JVP/VJP when possible	backprop, flows

Hessian Basics

1. Overview

The Hessian is the matrix of second partial derivatives of a scalar function. It describes local curvature and helps distinguish minima, maxima, and saddles.

2. Intuition

Gradient says which direction slopes; Hessian says how the surface bends. A bowl has positive curvature, an upside-down bowl negative curvature, and a saddle has mixed curvature.

3. Prerequisites

Partial derivatives, gradients, matrices, eigenvalues, and optimization.

4. Core Concepts

Subtopic	Meaning	Interview angle
Definition	$H_{ij} = \partial^2 f / (\partial x_i \partial x_j)$	Shape $d \times d$ for $f: \mathbb{R}^d \rightarrow \mathbb{R}$.
Curvature test	positive definite H at stationary point $\rightarrow$ local min	Uses eigenvalues.
Taylor approximation	$f(x+\delta) \approx f(x) + g^T \delta + \frac{1}{2} \delta^T H \delta$	Second-order model.
Hessian-vector product	Hv	Enables scalable curvature methods.

5. Algorithm / Working Process

Compute gradient; at a candidate stationary point examine Hessian eigenvalues; use a Newton-like step when appropriate, usually solving $H\delta = -g$ rather than inverting H .

6. Mathematical Foundation

For $f(x,y) = x^2 + 3xy + y^2$, $H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$. At $\nabla f = 0$: positive-definite Hessian means local minimum, negative-definite local maximum, and indefinite Hessian a saddle. Newton update: $\theta \leftarrow \theta - H^{-1} \nabla L$.

7. Practical Implementation

```
import torch

def f(v): return v[0]**2 + 3*v[0]*v[1] + v[1]**2
```

```
v = torch.tensor([1.0, 2.0])
H = torch.autograd.functional.hessian(f, v)
print(H) # tensor([[2., 3.], [3., 2.]])
```

8. Code Explanation

The Hessian's diagonal holds pure second derivatives; off-diagonal entries show how the two variables interact.

9. Training / Evaluation

Deep-learning training normally uses first-order optimizers because full Hessians are too large. Use curvature diagnostics, damping, or Hessian-vector methods in small/high-stakes optimization; evaluate validation metrics as usual.

10. Complexity and Cost

A full Hessian for d parameters needs $O(d^2)$ memory and generally expensive computation. Hessian-vector products avoid storing it and are often practical.

11. Common Use Cases

Newton methods, uncertainty approximations, curvature diagnostics, second-order least squares, and research on loss landscapes.

12. Common Mistakes

- Using Hessian test away from a stationary point as a min/max proof.
- Explicitly inverting Hessian.
- Assuming all Hessians are positive definite.
- Trying to materialize Hessian for large networks.

13. Edge Cases / Limitations

Indefinite/noisy Hessians make Newton steps unstable; ReLU networks are non-smooth; huge parameter counts rule out dense Hessians.

14. Variations

Diagonal Hessian approximations, Gauss–Newton, Fisher information, quasi-Newton (L-BFGS), and HVPs. L-BFGS/Newton basics are useful for placements and research.

15. Related Topics

Hessian is Jacobian of gradient; eigenvalues classify curvature; gradients provide first-order updates; Taylor expansion joins both.

16. Interview Questions

1. **What is Hessian?** Matrix of second partial derivatives.
2. **Hessian shape for d parameters?** $d \times d$.
3. **Positive definite at stationary point?** Local minimum.
4. **Indefinite Hessian?** Saddle point.
5. **Why seldom full Hessian in deep learning?** Quadratic memory/compute.
6. **Newton update?** $\theta - H^{-1} \nabla L$.
7. **Why solve instead of invert?** Better numerical stability and cost.
8. **What is HVP?** Hessian times vector without materializing Hessian.
9. **What do off-diagonals mean?** Cross-parameter curvature/interactions.
10. **Hessian vs Fisher?** Fisher is a positive-semidefinite curvature-related approximation often used probabilistically.

17. Practice Tasks

- Compute a 2-D Hessian by hand.
- Use eigenvalues to classify three quadratic surfaces.
- Compare gradient descent and Newton on a quadratic.
- Implement an HVP with autograd.

18. Project Ideas

Project	Stack / data	Resume value
Loss-curvature explorer	PyTorch, toy classifier	Second-order intuition.
Newton vs SGD benchmark	NumPy/sklearn, regression	Optimization comparison.
Laplace uncertainty demo	PyTorch, small classifier	Research-level uncertainty link.

19. Quick Revision

Formula: $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. **Use:** curvature. **Cost:** $O(d^2)$ memory. **Trap:** do not invert/materialize blindly.
One-liner: Hessian tells whether local terrain is bowl, hill, or saddle.

20. Final Cheat Sheet

Definition	Input $\rightarrow$ output	Pros	Cons
Hessian	scalar loss $\rightarrow d \times d$ curvature	rich second-order information	prohibitive at large d